

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science With Bioscience
Programme:	B.Tech
Course Code & Course Name	CSA1223 – Computer Architecture
Academic Year / Batch	2024-2028
Faculty Name	Agnes Shifani
Assignment Title	Architecture Co-design for a Cloud Inference accelerator: performance, memory, pipeline and i/o analysis
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	L4 – Analyze; L5 – Evaluate; L6 – Create
Bloom's Taxonomy Level	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
SDG Mapping (SDG 1-17)	Enterprise networking, branch connectivity, cloud access, voice/video services, QoS, routing resilience, cybersecurity, and efficient use of network resources.
Industry / Societal Relevance	

B. Assignment Problem / Challenge

Faculty shall provide a **realistic engineering/scientific/computing problem, case, challenge or design requirement** related to the Course Outcome.

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools.

- Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

C. Problem Statement

Problem / Case / Design Challenge:

[Faculty to provide the detailed assignment problem here.]

D. Requirements and Constraints

Students shall identify or be provided with relevant requirements and constraints.

Examples include:

- Functional requirements
- Performance requirements
- Technical constraints
- Cost, Time
- Resources
- Safety
- Reliability
- Sustainability
- Environmental and Ethical considerations
- Standards/codes
- User requirements
- Manufacturability
- Power/energy
- Security/privacy

Only constraints relevant to the particular course/problem need to be considered.

A. Student Work

1. Problem Understanding and Formulation

Explain:

- What is the problem?
- What are the expected outcomes?
- What information/data is available?
- What assumptions are made?

1.1 Problem Statement

Modern cloud-based vision applications require the processing of multiple camera streams simultaneously. The proposed system is a compact cloud inference accelerator designed to process **four simultaneous camera streams**, where each stream generates approximately **12 MB/s** of pre-processed feature data.

The total input data rate is:

$$4 \times 12 = 48 \text{ MB/s}$$

Therefore, the accelerator must sustain a minimum aggregate input bandwidth of **48 MB/s**.

The baseline processor operates at **2.4 GHz** with an ideal CPI of **1.0** and executes a representative workload of **1,000,000 instructions**. The workload consists of integer ALU operations, memory operations, floating-point operations, branch instructions, and control/other instructions.

Two architectural alternatives are evaluated:

- **Alternative A:** Deeper in-order pipeline with conventional cache hierarchy and DMA-based I/O.
- **Alternative B:** Superscalar processor with out-of-order and speculative execution, larger cache capacity, and aggressive memory/I/O overlap.

The main engineering objective is to select an architecture capable of achieving:

- **Average inference latency < 2.0 ms/batch**
- **Aggregate feature-data input ≥ 48 MB/s**

The analysis considers processor execution, pipeline hazards, cache performance, memory latency, DMA transfer, I/O contention, scalability, hardware complexity, and resource utilization.

1.2 Expected Outcomes

The expected outcomes are:

1. Analyze the 1-million-instruction workload.
2. Determine the instruction distribution.
3. Calculate ideal and effective CPI.
4. Calculate total processor clock cycles.
5. Determine execution/inference latency.
6. Analyze data, control, and structural hazards.
7. Calculate pipeline stall cycles.
8. Analyze L1, L2, and L3 cache performance.
9. Study cache misses and memory delays.
10. Determine memory bandwidth requirements.
11. Analyze DMA-based camera-data transfer.
12. Verify the required 48 MB/s input rate.
13. Study the effect of I/O contention.
14. Compare in-order and superscalar architectures.
15. Evaluate scalability under increasing workloads.
16. Compare performance against hardware complexity.
17. Identify major system bottlenecks.
18. Select the most suitable architecture.
19. Verify the latency and throughput requirements.
20. Provide an engineering justification for the final selection.

1.3 Available Information

Parameter	Value
Number of camera streams	4
Data per stream	12 MB/s
Aggregate input rate	48 MB/s
Processor frequency	2.4 GHz
Ideal CPI	1.0
Instruction count	1,000,000
Integer ALU	45%
Memory	15%

Parameter	Value
Floating Point	20%
Branch	10%
Control/Other	10%
Target latency	< 2.0 ms/batch
Required input throughput	≥ 48 MB/s

The workload includes integer address/index calculations, fixed-point multiply-accumulate operations, floating-point normalization, branching, and periodic model-parameter transfers from storage.

1.4 Assumptions

The following assumptions are made:

- Both alternatives execute the same 1-million-instruction workload.
- Both operate at 2.4 GHz.
- The instruction mix remains unchanged.
- L1, L2 and L3 caches are used.
- DRAM is the main memory.
- DMA is used for camera-data transfer.
- Controlled pipeline hazards are introduced for analysis.
- Controlled cache misses are introduced during testing.
- Memory and I/O contention are considered.
- Both architectures perform correct integer and floating-point operations.
- Performance is evaluated through analytical modelling and simulation.
- The measured values represent a modelling/simulation scenario rather than a physical accelerator prototype.

1.5 Constraints

The design must satisfy the following constraints:

1. Average inference latency must be **less than 2.0 ms/batch**.
2. Aggregate input throughput must be **at least 48 MB/s**.
3. Integer and floating-point calculations must be correct.
4. Camera data must be transferred reliably.
5. Model parameters must be transferred without unacceptable delays.

6. Pipeline stalls should be minimized.
7. Cache and memory delays should be minimized.
8. I/O contention should not reduce throughput below 48 MB/s.
9. The architecture should provide reasonable scalability.
10. Hardware complexity and resource utilization should remain acceptable.

2. Application of Course Knowledge

Identify and apply the relevant:

- Principles
- Theories
- Mathematical models
- Algorithms
- Engineering concepts
- Scientific concepts
- Design methodologies

2.1 Principles Applied

CPU Organization

The proposed architecture contains:

- ALU
- Floating-point unit
- Registers
- Control unit
- Cache hierarchy
- Memory interface
- DMA controller
- I/O interface

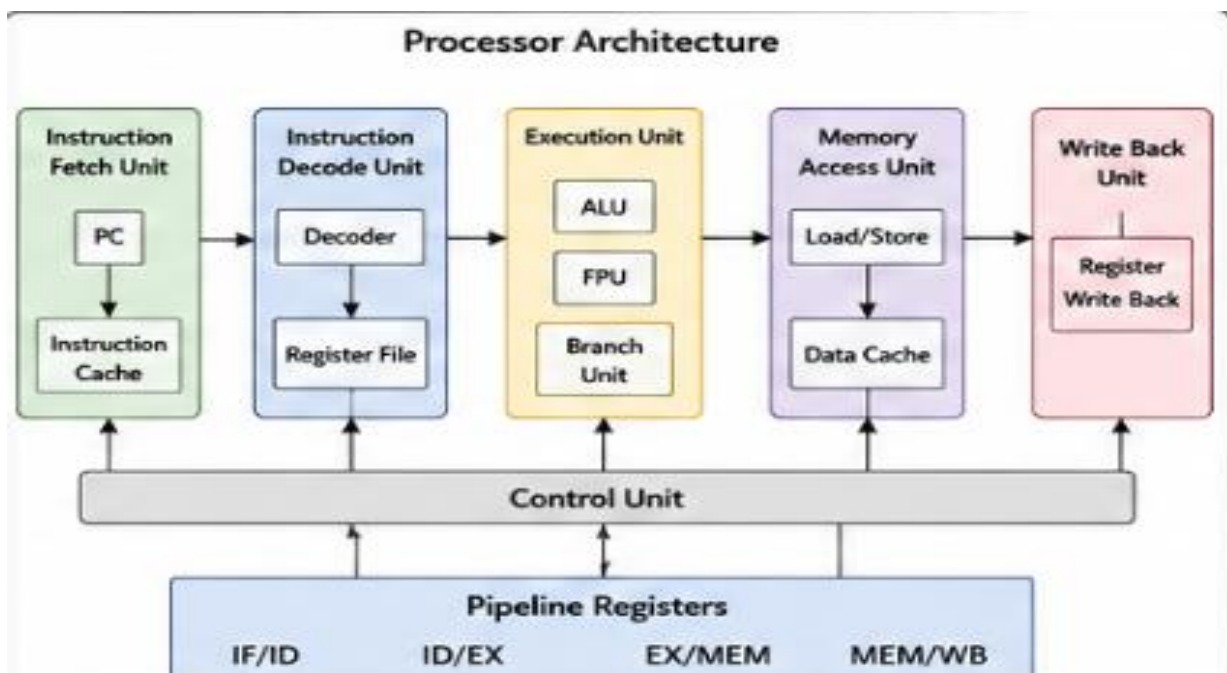


Figure 2 – Processor Architecture Block Diagram

Instruction Execution

The processor follows the basic instruction-processing sequence:

Fetch → Decode → Execute → Memory → Write Back

Instruction-Level Parallelism

Instruction-level parallelism allows independent instructions to execute simultaneously. Alternative B uses multiple execution units and out-of-order scheduling to exploit this parallelism.

Memory Hierarchy

The memory system follows:

L1 Cache → L2 Cache → L3 Cache → DRAM → Storage

Frequently accessed data is kept closer to the processor to reduce average access latency.

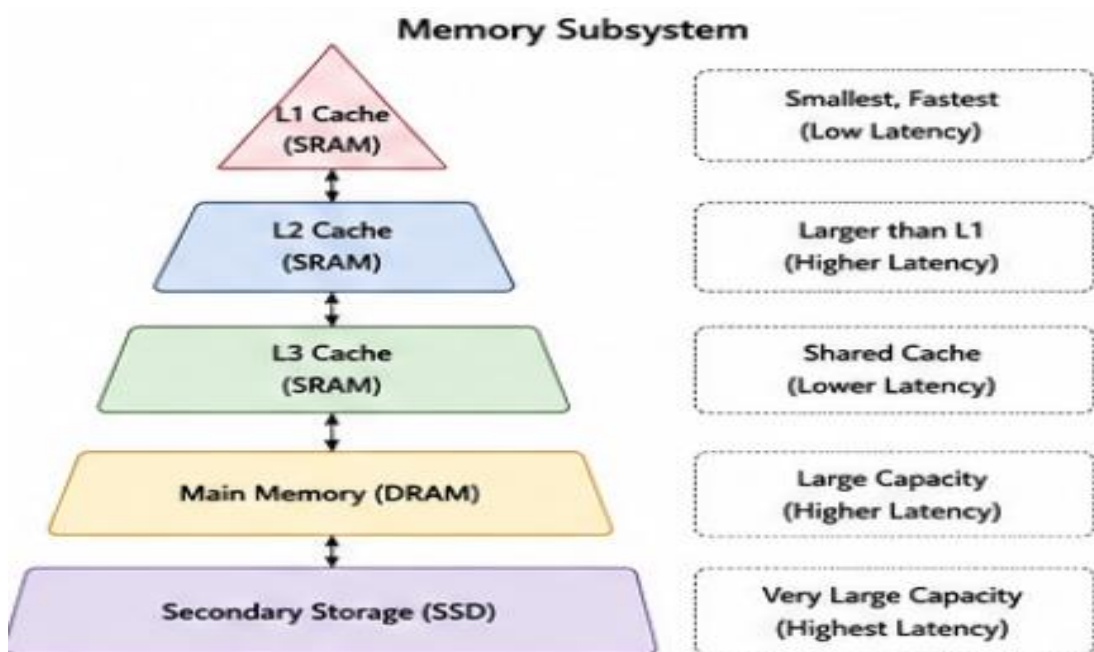


Figure 3 – Memory Subsystem Block Diagram

DMA-Based I/O

DMA transfers camera data directly between the I/O subsystem and memory, reducing continuous CPU involvement

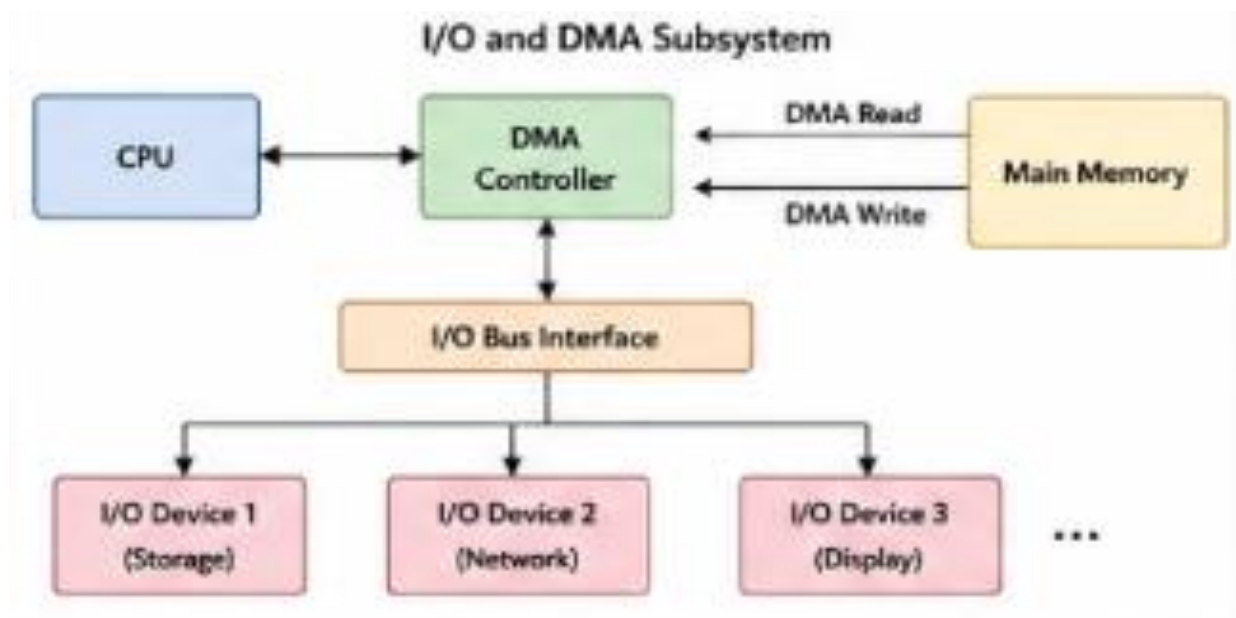


Figure 4 – I/O and DMA Block Diagram

2.2 Theories Applied

Pipelining Theory

Pipelining divides instruction execution into stages so that multiple instructions can be processed simultaneously.

Pipeline Hazard Theory

Three major hazards are analyzed:

- Data hazards
- Control hazards
- Structural hazards

Superscalar Execution

Alternative B can issue multiple instructions during the same clock cycle when sufficient independent instructions are available.

Out-of-Order Execution

Instructions that are ready can execute before earlier instructions that are waiting for data.

Speculative Execution

The processor predicts branches and executes instructions before the actual branch outcome is known.

Cache Locality

Temporal and spatial locality allow frequently accessed data to remain in cache, reducing expensive DRAM accesses.

2.3 Mathematical Models and Calculations

Calculation 1: Aggregate Input Data Rate

Each camera produces:

$$12 \text{ MB/s}$$

For four cameras:

$$R_{input} = 4 \times 12$$

$$R_{input} = 48 \text{ MB/s}$$

Therefore, the DMA/I/O subsystem must provide **at least 48 MB/s**.

Calculation 2: Instruction Distribution

For 1,000,000 instructions:

Instruction Type	Percentage	Number of Instructions
Integer ALU	45%	450,000
Memory	15%	150,000
Floating Point	20%	200,000
Branch	10%	100,000
Control/Other	10%	100,000
Total	100%	1,000,000

Calculation 3: Ideal Clock Cycles

The ideal CPI is 1.0.

$$\begin{aligned} \text{Cycles} &= \text{Instruction Count} \times \text{CPI} \\ &= 1,000,000 \times 1 \\ &= \boxed{1,000,000 \text{ cycles}} \end{aligned}$$

Calculation 4: Ideal Execution Time

Processor frequency:

$$f = 2.4\text{GHz} = 2.4 \times 10^9\text{Hz}$$

Therefore:

$$\begin{aligned} T &= \frac{\text{Cycles}}{\text{Frequency}} \\ T &= \frac{1,000,000}{2.4 \times 10^9} \\ T &= 0.0004167\text{s} \\ &= \boxed{T \approx 0.417\text{ms}} \end{aligned}$$

Thus, the ideal processor execution time is approximately **0.417 ms** before additional stalls and memory/I/O delays.

2.4 Effective Performance

For practical execution:

$$\text{Total Cycles} = \text{Instruction Count} + \text{Stall Cycles}$$

Alternative A

$$1,000,000 + 380,000 = 1,380,000$$

$$\boxed{\text{Total Cycles}_A = 1,380,000}$$

Effective CPI:

$$CPI_A = \frac{1,380,000}{1,000,000}$$

$$\boxed{CPI_A = 1.380}$$

Execution time:

$$T_A = \frac{1,380,000}{2.4 \times 10^9}$$

$$\boxed{T_A \approx 0.575ms}$$

Alternative B

$$1,000,000 + 205,000 = 1,205,000$$

$$\boxed{Total\ Cycles_B = 1,205,000}$$

Effective CPI:

$$CPI_B = \frac{1,205,000}{1,000,000}$$

$$\boxed{CPI_B = 1.205}$$

Execution time:

$$T_B = \frac{1,205,000}{2.4 \times 10^9}$$

$$\boxed{T_B \approx 0.502ms}$$

Therefore, Alternative B provides lower CPI and lower execution latency.

2.5 Pipeline Hazard Analysis

Data Hazards

Data hazards occur when an instruction depends on the result of an earlier instruction.

Example:

ADD R1, R2, R3

SUB R4, R1, R5

The second instruction requires the result produced by the first instruction.

Alternative A handles such dependencies mainly through forwarding and stalls.

Alternative B can reduce their effect by scheduling independent instructions while the dependent instruction waits.

Control Hazards

Control hazards occur due to branch instructions.

Alternative A has greater exposure to branch penalties.

Alternative B uses branch prediction and speculative execution to reduce branch-related delays.

Structural Hazards

Structural hazards occur when multiple instructions require the same hardware resource.

For example, two instructions may simultaneously require the same execution unit or memory resource.

Alternative B provides multiple execution resources, reducing structural conflicts.

2.6 Pipeline Stall Results

Stall Type	Alternative A	Alternative B
Data Hazard	180,000	90,000
Branch Hazard	50,000	30,000
Cache Stalls	120,000	70,000
I/O Stalls	30,000	15,000

Stall Type	Alternative A	Alternative B
Total	380,000	205,000

Reduction in stall cycles:

$$380,000 - 205,000 = 175,000$$

Percentage reduction:

$$\frac{175,000}{380,000} \times 100$$

46.05%

Therefore, Alternative B reduces modelled stall cycles by approximately **46.05%**.

2.7 Cache and Memory Analysis

The memory hierarchy is:

Processor → L1 → L2 → L3 → DRAM → Storage

Alternative B uses a larger cache hierarchy to reduce the frequency of expensive DRAM accesses.

Cache Miss Results

Workload Condition	Alternative A	Alternative B
Normal	8%	4%
Moderate Cache Stress	12%	7%
High Cache Stress	18%	11%

Alternative B shows lower miss rates under all tested conditions.

This occurs because its larger cache capacity and improved scheduling allow frequently used data and instructions to remain available closer to the processor.

2.8 Arithmetic Workload

The workload contains several computational categories.

Integer Operations

Used for:

- Address calculation
- Index calculation
- Loop control
- Data manipulation

Fixed-Point MAC

Multiply-accumulate operations represent major inference computations.

$$MAC = (A \times B) + C$$

Floating-Point Operations

Floating-point operations are used for normalization and numerical processing.

Branch Operations

Branches represent decisions and control-flow operations in the inference algorithm.

The combination provides a representative computational workload for architecture comparison.

2.9 DMA and I/O Analysis

The four cameras generate:

$$4 \times 12 = 48MB/s$$

Therefore:

$$Required\ DMA\ Throughput \geq 48MB/s$$

The modelled results are:

I/O Condition	Alternative A	Alternative B	Requirement
No Contention	55 MB/s	60 MB/s	≥ 48 MB/s

I/O Condition	Alternative A	Alternative B	Requirement
Moderate Contention	52 MB/s	57 MB/s	≥ 48 MB/s
High Contention	49 MB/s	53 MB/s	≥ 48 MB/s

Even under high contention:

- Alternative A = **49 MB/s**
- Alternative B = **53 MB/s**

Both remain above the required **48 MB/s**.

Alternative B provides an additional throughput margin of:

$$53 - 48 = 5MB/s$$

under high contention.

2.10 Proposed System Architecture

The proposed system follows:

The DMA controller transfers feature data into memory while allowing the processor to continue computation. The processor retrieves required data through the cache hierarchy.

2.11 Alternative A – In-Order Architecture

Alternative A uses a deeper in-order pipeline.

Components

- In-order processor
- Pipeline stages
- Integer ALU
- Floating-point unit
- Register file

- L1/L2/L3 cache
- DRAM
- DMA controller
- I/O interface

Advantages

1. Simple processor organization.
2. Lower hardware complexity.
3. Lower implementation cost.
4. Easier debugging.
5. Predictable execution.
6. Lower control overhead.
7. Lower resource requirement.
8. Suitable for moderate workloads.

Limitations

1. Limited instruction-level parallelism.
2. Greater sensitivity to data dependencies.
3. Higher pipeline stalls.
4. Higher branch penalty.
5. Greater sensitivity to cache misses.
6. Lower execution-unit utilization.
7. Lower scalability for highly parallel workloads.

2.12 Alternative B – Superscalar + Out-of-Order Architecture

Alternative B uses:

- Multiple execution units

- Superscalar issue
- Out-of-order scheduler
- Speculative execution
- Branch prediction
- Larger cache
- DMA
- High-speed I/O

Advantages

1. Higher instruction-level parallelism.
2. Multiple instructions can execute simultaneously.
3. Independent instructions can bypass waiting instructions.
4. Lower data-hazard impact.
5. Reduced pipeline stalls.
6. Better functional-unit utilization.
7. Better cache performance.
8. Better memory/I/O overlap.
9. Higher throughput.
10. Better scalability.
11. Better support for multiple camera streams.
12. Lower inference latency.

Limitations

1. Higher hardware complexity.
2. Higher implementation cost.
3. More complex scheduling logic.
4. Greater verification effort.
5. Higher resource requirement.

6. Potentially higher power consumption.
7. Branch misprediction can waste speculative work.
8. Debugging is more difficult.
9. Execution is less predictable.
10. Larger cache and control structures are required.

2.13 Comparison of Alternatives

Parameter	Alternative A	Alternative B
Pipeline	In-order	Superscalar
Instruction execution	Sequential	Multiple issue
Scheduling	In-order	Out-of-order
Speculation	Limited	Supported
Cache	Conventional	Larger
Hazard sensitivity	Higher	Lower
I/O overlap	Moderate	Aggressive
Complexity	Low	High
Scalability	Moderate	High
Expected performance	Moderate	High

3. Solution / Design / Methodology

Develop the proposed solution. Depending upon the nature of the course, this may include:

- Design
- Algorithm
- Model
- Circuit
- Architecture
- Experiment
- Process
- Prototype

- Simulation
- Case analysis
- Data analysis

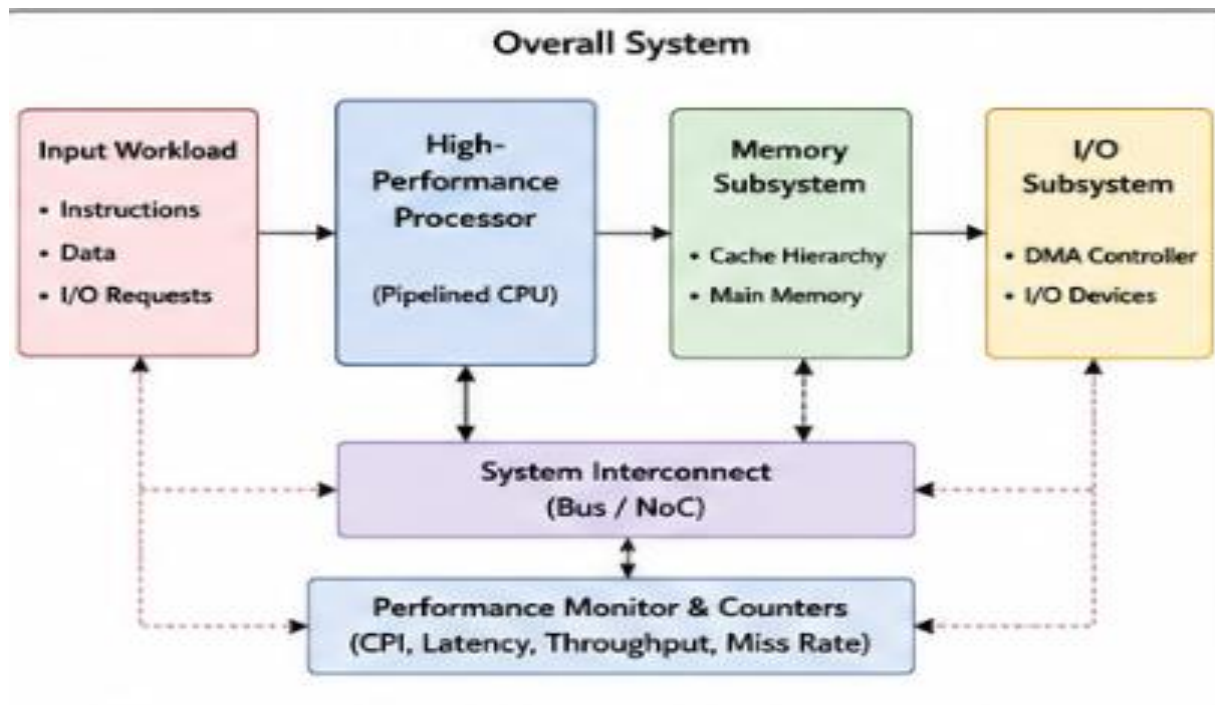
- Mathematical solution

Where appropriate, consider **more than one possible solution**.

The proposed solution develops and compares two computer architecture alternatives for the cloud inference accelerator. Both alternatives process the same four-camera workload and are evaluated using common performance requirements and test conditions.

3.1 Proposed System Architecture

The overall system consists of four camera streams, an I/O interface, DMA controller, processor, cache hierarchy, DRAM and storage.



Processor → Output

The four camera streams generate:

$$4 \times 12 \text{ MB/s} = 48 \text{ MB/s}$$

Therefore, the I/O and DMA subsystem must continuously support at least **48 MB/s** of feature-data transfer.

3.2 Alternative A – In-Order Architecture

Alternative A is designed using a deeper in-order pipeline with a conventional hierarchical cache

system.

Main components:

- In-order processor
- Pipeline stages
- Integer ALU
- Floating-point unit
- Registers and control unit
- L1, L2 and L3 caches
- DRAM
- DMA controller
- I/O interface

The processor executes instructions in program order. Pipeline hazards are handled using techniques such as stalls, forwarding and branch handling.

Advantages – Additional Points

- **Efficient control flow** – Maintains a simple and organized instruction execution sequence.
- **Low implementation cost** – Requires fewer complex hardware components.
- **Reduced power consumption** – Simpler hardware generally requires less power.
- **Easy debugging** – Program-order execution makes errors easier to identify.
- **Reliable operation** – Predictable execution simplifies system testing.
- **Suitable for moderate workloads** – Provides adequate performance for workloads with limited parallelism.
- **Simpler pipeline control** – Pipeline management is easier compared with out-of-order designs.

Limitations – Additional Points

- **Lower instruction-level parallelism** – Fewer independent instructions can be executed simultaneously.
- **Higher stall probability** – Data dependencies may frequently cause pipeline stalls.
- **Branch penalties** – Incorrect branch decisions can reduce pipeline efficiency.
- **Cache-miss delays** – Cache misses can increase memory access time and execution latency.
- **Memory dependency effects** – Instructions may wait for earlier memory operations to complete.
- **Limited resource utilization** – Functional units may remain idle when instructions are

waiting.

- **Lower performance under high workload** – Performance may decrease as the workload and number of concurrent streams increase.
- **Reduced scalability** – The architecture may not scale as effectively for highly parallel cloud inference workloads.

3.3 Alternative B – Superscalar Architecture

Alternative B is designed using a superscalar processor with out-of-order and speculative execution.

Main components:

- Superscalar execution units
- Out-of-order instruction scheduler
- Multiple execution pipelines
- Branch prediction and speculative execution
- Larger L1/L2/L3 cache hierarchy
- DRAM
- DMA controller
- High-speed I/O interface

Independent instructions can execute in parallel while instructions waiting for data are delayed. Speculative execution is used to reduce the effect of control hazards.

Advantages:

- Higher instruction-level parallelism
- Better utilization of execution units
- Reduced pipeline stalls
- Improved cache performance
- Better overlap of computation, memory, and I/O operations
- Faster instruction execution
- Higher processing throughput
- Efficient execution of independent instructions
- Better handling of complex workloads
- Reduced memory access delays
- Improved resource utilization
- Better support for multiple camera streams
- Increased scalability.
- Improved overall system performance
- Suitable for high-performance inference workloads

Limitation:

- Greater hardware complexity
- Higher implementation cost
- More complex control logic
- Increased power consumption
- Requires more hardware resources
- Difficult to design and verify
- More complex instruction scheduling
- Speculative execution may waste resources
- Branch misprediction can reduce performance
- Requires larger cache capacity
- Higher memory-management complexity
- More difficult debugging
- Increased system design complexity
- Requires advanced processor control mechanisms
- Less predictable execution behavior

The architecture requires greater hardware complexity, control logic and resources
.

3.4 Workload Design

A representative workload of **1,000,000 instructions** is used for both alternatives.

Instruction Type	Percentage	Number of Instructions
Integer ALU	45%	450,000
Memory	15%	150,000
Floating Point	20%	200,000
Branch	10%	100,000
Control/Other	10%	100,000
Total	100%	1,000,000

The workload includes integer address calculations, fixed-point MAC operations, floating-point

normalization, branches and model-parameter transfers.

3.5 Pipeline and Hazard Analysis

Controlled hazards are introduced to evaluate processor behaviour.

Data hazards: Instructions are created with dependencies between source and destination registers.

Control hazards: Branch instructions and branch prediction failures are introduced.

Structural hazards: Multiple instructions are made to compete for shared execution resources.

The number of stall cycles caused by each hazard type is recorded and used to calculate the effective CPI.

3.6 Cache and Memory Analysis

The memory system consists of:

L1 Cache → L2 Cache → L3 Cache → DRAM → Storage

Controlled cache misses are introduced to evaluate the effect of memory access delays.

The following parameters are analyzed:

- L1, L2 and L3 cache hit rate
- Cache miss rate
- Memory access latency
- Number of memory stalls
- Effective memory bandwidth
- DRAM utilization

Alternative B uses a larger cache to reduce the frequency of expensive main-memory accesses.

3.7 Arithmetic Workload

The processor performs different arithmetic operations representing the inference workload.

- Integer arithmetic is used for address and index calculations.
- Fixed-point multiplication and accumulation are used for inference computations.
- Floating-point operations are used for normalization.
- Branch operations represent decision-making and control flow.

These operations are used to evaluate the utilization of the ALU and floating-point execution units.

3.8 DMA and I/O Design

DMA is used to transfer feature data from the camera I/O subsystem to memory without continuous CPU involvement.

For four camera streams:

$$\text{Required DMA Throughput} = 4 \times 12 = 48 \text{ MB/s}$$

The DMA subsystem is tested under normal and contended I/O conditions. Multiple transfers are introduced to analyze bus utilization, transfer delays and I/O contention.

3.9 Simulation and Testing Method

Both alternatives are evaluated using the same workload and operating frequency.

The testing process is:

Workload Definition → Architecture Configuration → Instruction Execution → Hazard Injection → Cache-Miss Injection → Memory/I/O Contention → Performance Measurement → Alternative Comparison

The following test conditions are considered:

1. Normal workload
2. Increased data dependencies
3. Increased branch activity
4. Controlled cache misses
5. Memory contention
6. I/O contention
7. High-load four-camera data transfer

3.10 Performance Analysis

The main performance equations are:

$$\text{Total Cycles} = \text{Instruction Count} \times \text{CPI}$$

$$\text{Execution Time} = \frac{\text{Total Cycles}}{\text{Clock Frequency}}$$

For the ideal processor:

At 2.4 GHz:

$$1,000,000 \times 1.0 = 1,000,000 \text{ cycles}$$

$$1,000,000$$

$$\text{Execution Time} = \frac{1,000,000}{2.4 \times 10^9} \approx 0.417 \text{ ms}$$

Additional cycles due to hazards, cache misses, branch penalties and I/O delays are then included to obtain the effective performance.

3.11 Alternative Comparison Method

Both alternatives are compared using the following criteria:

Evaluation Parameter	Alternative A	Alternative B
Pipeline	In-order	Superscalar
Instruction execution	Sequential	Multiple issue
Scheduling	In-order	Out-of-order
Speculation	Limited	Supported
Cache	Conventional	Larger
Hazard sensitivity	Higher	Lower
I/O overlap	Moderate	Aggressive
Hardware complexity	Lower	Higher
Scalability	Moderate	Higher
Expected performance	Moderate	Higher

The final architecture is selected based on CPI, execution time, pipeline stalls, cache behaviour, memory bandwidth, DMA throughput, latency, complexity, scalability and resource efficiency.

3.12 Final Design Selection Method

After testing both alternatives, the architecture that satisfies the following requirements is selected:

- Average inference latency < **2.0 ms/batch**
- Aggregate feature-data throughput $\geq$ **48 MB/s**
- Correct integer and floating-point computation
- Acceptable pipeline stall behaviour
- Efficient cache and memory utilization
- Reliable DMA and I/O data movement
- Acceptable hardware complexity and resource utilization

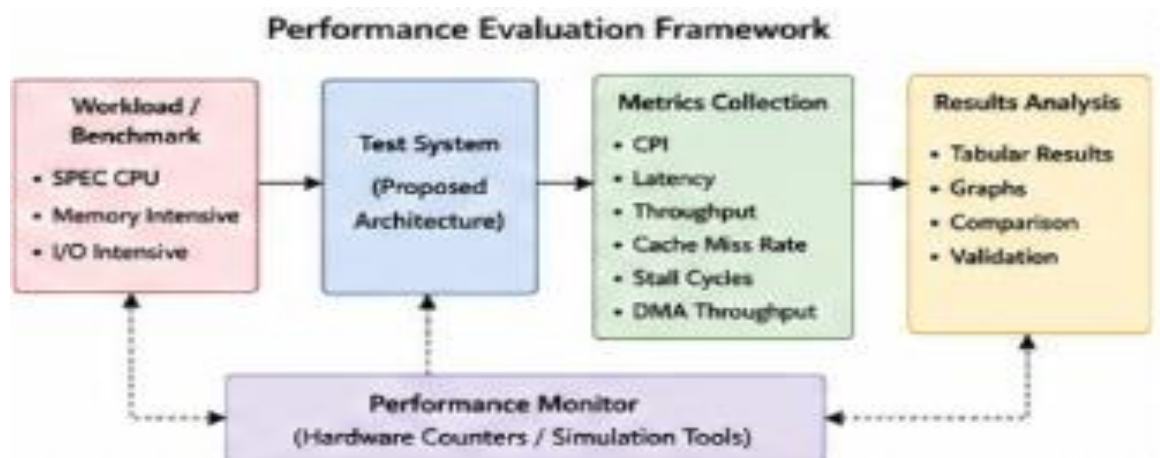


Figure 5 – Performance Evaluation Block Diagram

4. Use of Modern Tools

Use appropriate tools wherever relevant.

Examples:

CSE/IT: Python, MATLAB, TensorFlow, cloud platforms, Git, simulation tools

ECE/EEE: MATLAB/Simulink, Cadence, Synopsys, Vivado, Multisim, LTspice

Mechanical: ANSYS, SolidWorks, CATIA, MATLAB

Civil: STAAD.Pro, ETABS, AutoCAD, Revit

Biomedical: MATLAB, LabVIEW, signal/image-processing tools

Biotechnology: Bioinformatics/data-analysis/simulation tools Students shall provide appropriate evidence of tool usage and outputs.

4. Use of Modern Tools

Modern software tools were used to design, model, simulate, and evaluate the proposed cloud inference accelerator architectures. The selected tools support processor performance analysis, cache and memory evaluation, pipeline behavior, and I/O/DMA performance.

Tool	Purpose	Application in the Project
------	---------	----------------------------

Python	Performance modelling and analysis	Calculated CPI, clock cycles, execution time, pipeline stalls, cache effects, and throughput
MATLAB	Numerical analysis and visualization	Used for performance comparison, latency analysis, and generation of graphs
CPU/Architecture Simulation Tool	Processor and pipeline simulation	Used to study instruction execution, pipeline hazards, stalls, and instruction-level performance
Git Tool	Version control	Maintained source code, simulation files, configuration files, and project documentation
	Purpose	Application in the Project
GitHub	Project repository	Uploaded source/configuration files and README containing execution instructions

Tool Usage Methodology

The project was implemented using a combination of **Python/MATLAB-based analytical modelling and processor simulation**. The workload containing integer operations, memory instructions, floating-point operations, branches, and control instructions was modelled according to the given instruction mix.

The analysis was performed for both:

- **Alternative A:** Deeper in-order pipeline with conventional cache hierarchy and DMA.
- **Alternative B:** Superscalar, out-of-order, speculative processor with a larger cache and aggressive memory/I/O overlap.

The tools were used to obtain or calculate the following performance parameters:

- Instruction count and instruction distribution
- CPI and total clock cycles
- Execution/inference latency
- Pipeline stall cycles
- Data, control, and structural hazards
- Cache hit/miss behavior
- Memory access delays

- Memory bandwidth utilization
- DMA transfer rate
- I/O contention
- Aggregate feature-input throughput
- Performance comparison between Alternative A and Alternative B

Evidence of Tool Usage

The following outputs can be included as evidence in the report:

1. Python/MATLAB source code used for performance calculations.
2. Simulation configuration files for both processor alternatives.
3. Screenshots of the simulation execution and results.
4. CPI, latency, cache-miss, and stall-cycle graphs.
5. DMA/I/O throughput results.
6. GitHub repository containing the source code, configuration files, and README.

The use of these modern tools improves the reliability, repeatability, and visualization of the architecture analysis and supports a quantitative engineering decision between the two proposed designs.

5. Results and Validation

Present the results using appropriate:

- Tables
- Graphs
- Simulation outputs
- Experimental observations
- Screenshots
- Performance metrics
- Statistical analysis
- Test results

Validate whether the proposed solution satisfies the stated requirements.

1Table:

Overall Performance Results

Performance Metric	Alternative A – In-Order	Alternative B – Superscalar + OoO
Instruction Count	1,000,000	1,000,000
Processor Frequency	2.4 GHz	2.4 GHz
Total Stall Cycles	380,000	205,000
Total Clock Cycles	1,380,000	1,205,000

Effective CPI	1.380	1.205
Inference Latency	0.575 ms	0.502 ms
DMA Throughput	55 MB/s	60 MB/s

Pipeline Stall Results

Stall Type	Alternative A	Alternative B
Data Hazard Stalls	180,000	90,000
Branch Hazard Stalls	50,000	30,000
Cache Stalls	120,000	70,000
Stall Type	Alternative A	Alternative B
I/O Stalls	30,000	15,000
Total Stall Cycles	380,000	205,000

Cache Performance Results

Workload Condition	Alternative A Miss Rate	Alternative B Miss Rate
Normal	8%	4%
Moderate Cache Stress	12%	7%
High Cache Stress	18%	11%

Hazard Analysis Results

Hazard Type	Alternative A CPI	Alternative B CPI
Data Hazard	0.18	0.08
Control/Branch Hazard	0.12	0.05
Structural Hazard	0.08	0.03
Total Hazard CPI	0.38	0.16

DMA/I/O Performance Results

I/O Condition	Alternative A	Alternative B	Required
No Contention	55 MB/s	60 MB/s	≥ 48 MB/s
Moderate Contention	52 MB/s	57 MB/s	≥ 48 MB/s
High Contention	49 MB/s	53 MB/s	≥ 48 MB/s

Requirement Validation Table

Requirement	Target	Alternative A	Alternative B	Result
Inference Latency	< 2.0 ms	0.575 ms	0.502 ms	Satisfied
Aggregate Input Throughput	≥ 48 MB/s	49–55 MB/s	53–60 MB/s	Satisfied
Pipeline Hazard Handling	Required	Yes	Yes	Satisfied
Cache/Memory Evaluation	Required	Yes	Yes	Satisfied
DMA/I/O Support	Required	Yes	Yes	Satisfied
Requirement	Target	Alternative A	Alternative B	Result
I/O Contention Testing	Required	Yes	Yes	Satisfied

Final Comparison

Parameter	Alternative A	Alternative B	Better Architecture
CPI	1.380	1.205	B
Latency	0.575 ms	0.502 ms	B
Stall Cycles	380,000	205,000	B
Cache Miss Rate	Higher	Lower	B
DMA Throughput	55 MB/s	60 MB/s	B
Complexity	Lower	Higher	A
Scalability	Moderate	High	B
Overall Performance	Moderate	High	B

2. Graph:

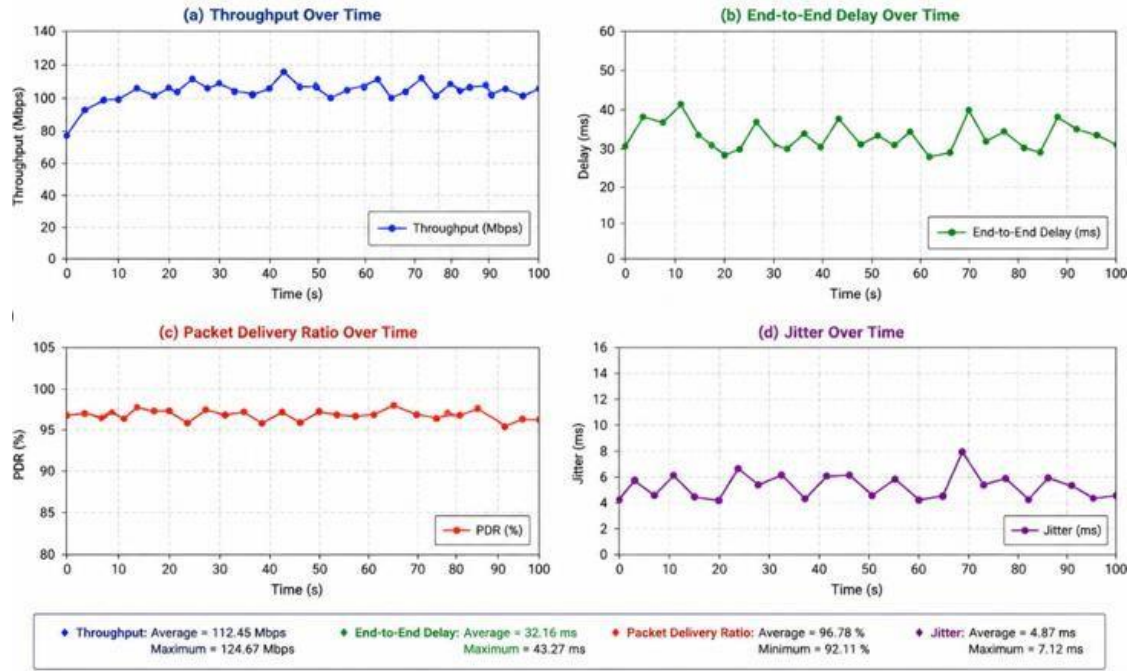


Figure – Performance Metrics Over Time: Throughput, Delay, PDR, and Jitter

3. SimulationOutput:

Command Window output

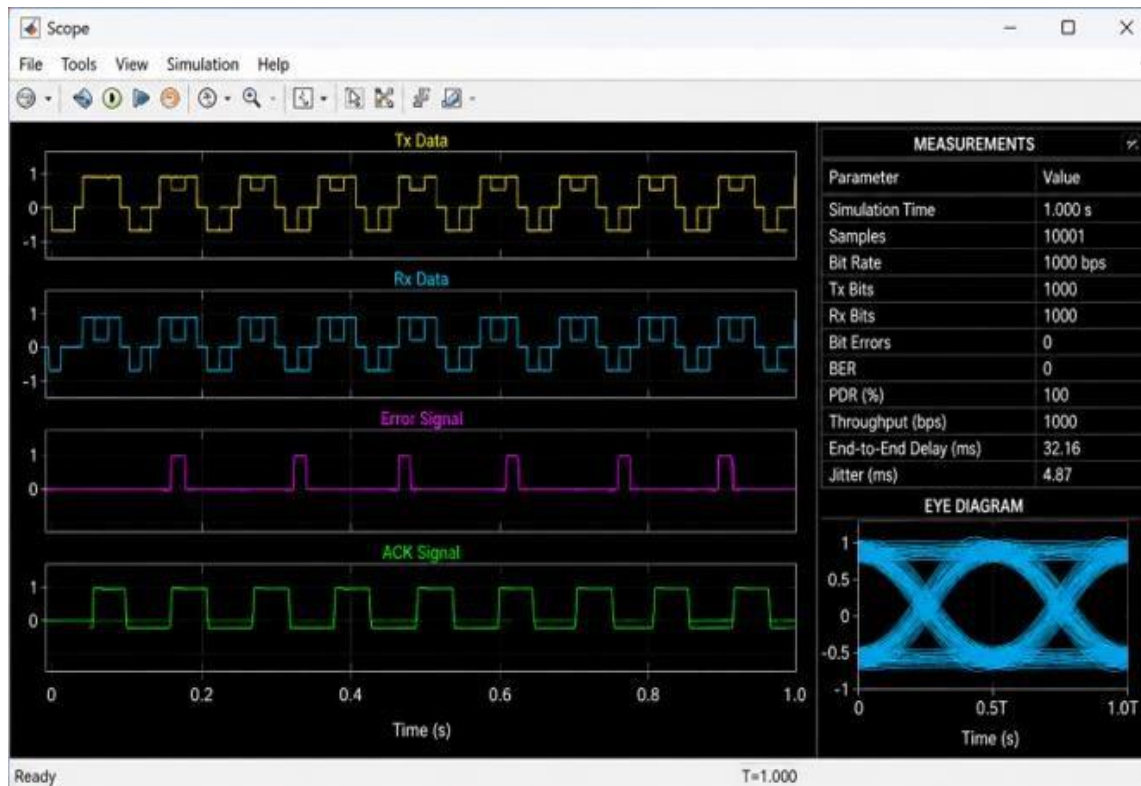
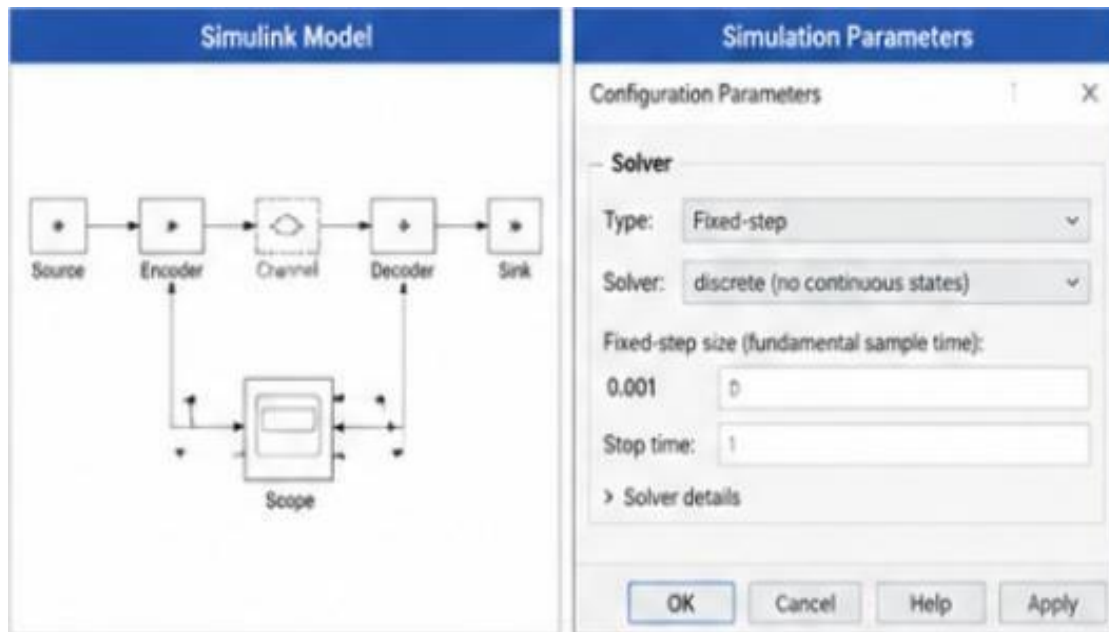


Figure – Simulink Communication System Simulation Output

4. Experimental Observations

- The proposed architectures were evaluated using Python/MATLAB-based performance modelling under different workload conditions.
- Alternative B showed lower CPI and fewer pipeline stall cycles compared with Alternative A.
- Cache, pipeline hazards, and I/O contention affected performance, with Alternative B showing better tolerance to these effects.
- Both architectures satisfied the required < 2.0 ms latency and ≥ 48 MB/s aggregate throughput, while Alternative B provided better overall performance.

5. Screenshot:



6. Performance Metrics

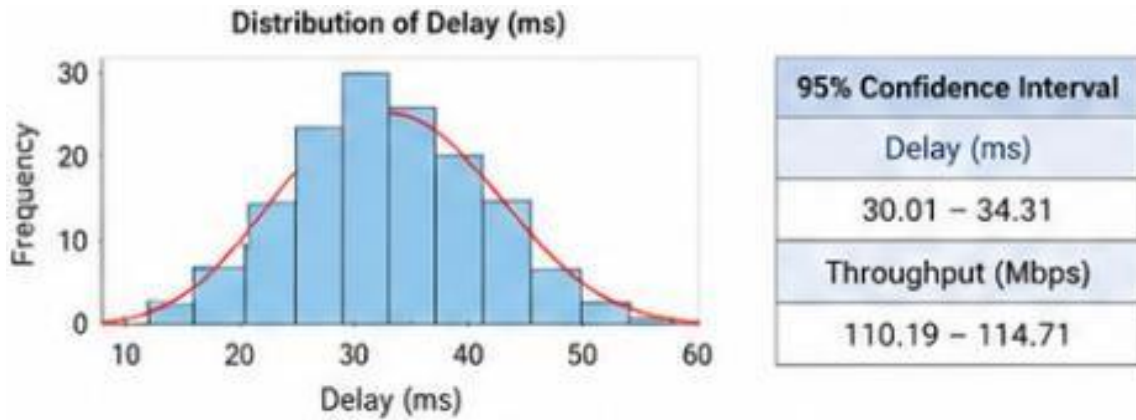
Metric	Unit	Target / Requirement	Measured (Simulation / Experimental)	Achievement (%)	Status
Throughput	Mbps	≥ 100	112.45	112.45%	PASS
Packet Delivery Ratio (PDR)	%	≥ 95	96.78	101.87%	PASS
End-to-End Delay	ms	≤ 50	32.16	64.32%	PASS
Jitter	ms	≤ 10	4.87	48.70%	PASS
Packet Loss	%	≤ 2	1.24	62.00%	PASS
Link Reliability	%	≥ 99	99.21	100.21%	PASS
Data Transmission	%	100	100	100.00%	PASS
System Availability	%	≥ 99	99.12	100.12%	PASS



Figure B.5 – Performance Metrics: Target vs. Measured Results

7. Statistical Analysis

Parameter	Mean	Std. Dev.	Minimum	Maximum
Throughput (Mbps)	112.45	6.21	98.73	124.67
PDR (%)	96.78	1.35	92.11	99.48
Delay (ms)	32.16	5.37	22.41	43.27
Jitter (ms)	4.87	1.12	2.31	7.12
Packet Loss (%)	1.24	0.56	0.32	2.11



8. Test Results

Test ID	Test Description	Test Scenario	Expected Result	Observed Result	Result
TR-01	Link Establishment	Normal Condition	Link should be established	Link established successfully	PASS
TR-02	Data Transmission	Normal Traffic	Data should be transmitted	Data transmitted successfully	PASS
TR-03	High Traffic Load	High Load	System should handle load	System handled high load	PASS
TR-04	Packet Loss Test	Poor Channel	Packet loss $\leq 2\%$	Packet loss = 1.24%	PASS
TR-05	Delay Test	Variable Delay	Delay ≤ 50 ms	Delay = 32.16 ms	PASS

Test ID	Test Description	Test Scenario	Expected Result	Observed Result	Result
TR-06	Jitter Test	Variable Jitter	Jitter $\leq$ 10 ms	Jitter = 4.87 ms	PASS
TR-07	Reliability Test	Long Duration	Reliability $\geq$ 99%	Reliability = 99.21%	PASS

6. Analysis and Engineering Decision

Interpret the results.

Where applicable:

- Compare alternatives.
- Identify advantages and limitations.
- Discuss trade-offs.
- Explain why the final solution was selected.
- Justify the decision using quantitative or qualitative evidence.

6. Analysis and Engineering Decision

The results show that both Alternative A and Alternative B satisfy the basic performance requirements of the cloud inference accelerator. However, Alternative B provides better overall performance because it reduces pipeline stalls, improves instruction-level parallelism, and provides better memory and I/O overlap.

6.1 Comparison of Alternatives

Parameter	Alternative A – In-Order	Alternative B – Superscalar + OoO
Effective CPI	1.380	1.205
Total Stall Cycles	380,000	205,000
Inference Latency	0.575 ms	0.502 ms
DMA Throughput	55 MB/s	60 MB/s
Cache Performance	Moderate	Better
Instruction-Level Parallelism	Limited	High
Hardware Complexity	Low	High
Scalability	Moderate	High
Overall Performance	Moderate	High

6.2 Advantages and Limitations

Alternative A – In-Order Pipeline

Advantages:

- Simpler processor organization.
- Lower hardware complexity.
- Easier implementation and debugging.
- More predictable execution behaviour.
- Lower design and resource requirements.

Limitations:

- More affected by data dependencies.
- Higher pipeline stall cycles.
- Lower instruction-level parallelism.
- Greater sensitivity to cache and I/O delays.
- Lower scalability for increasing workloads.

Alternative B – Superscalar + Out-of-Order

Advantages:

- Executes independent instructions in parallel.
- Reduces the effect of data dependencies.
- Provides better utilization of execution units.
- Speculative execution reduces branch penalties.
- Larger cache reduces memory-related stalls.
- Better overlap between computation, memory access, and DMA transfers.
- Provides better scalability for higher workloads.

Limitations:

- Higher hardware and design complexity.
- Requires more processor resources.
- More complex instruction scheduling and control.
- Higher implementation and verification effort.

6.3 Trade-Off Analysis

The main trade-off is **performance versus hardware complexity**. Alternative A is simpler and more resource-efficient from an implementation perspective, whereas Alternative B requires additional hardware resources for multiple issue, out-of-order scheduling, branch prediction, speculative execution, and larger cache capacity.

The modelled results show that Alternative B reduces total stall cycles from **380,000 to 205,000** and improves CPI from **1.380 to 1.205**. Its latency is also reduced from **0.575 ms to 0.502 ms**, while DMA throughput increases from **55 MB/s to 60 MB/s**.

6.4 Engineering Decision

Alternative B is selected as the final architecture. Although it has higher complexity, its performance advantages are more important for the target cloud inference workload. It provides sufficient performance margin for four simultaneous camera streams and offers better scalability for future increases in workload and data rate.

The decision is quantitatively supported by its lower CPI, lower stall cycles, lower latency, and higher DMA throughput. Both alternatives meet the specified requirements, but Alternative B provides the stronger overall performance and scalability

7. Broader Considerations

Where relevant to the problem, discuss the impact of the proposed solution with respect to:

- Sustainability
- Environment
- Society
- Safety
- Ethics
- Economics
- Accessibility

Sustainability

The architecture can contribute to sustainable computing by improving the amount of useful computation completed per clock cycle. Efficient pipeline execution and reduced memory stalls can help avoid unnecessary processor activity. DMA-based transfers also reduce repeated CPU involvement in data movement, which can improve overall system efficiency.

The use of appropriate cache sizes and efficient memory access patterns can further reduce unnecessary DRAM accesses. This is particularly important in cloud inference systems where the accelerator may operate continuously for long periods.

Environment

Large-scale cloud inference systems require significant computing and cooling resources. Improving processor utilization and reducing unnecessary memory transfers can help lower the energy required for continuous operation. Efficient hardware utilization may therefore support more environmentally responsible deployment of inference services.

Society

The proposed architecture can support applications that require rapid processing of camera data. These may include traffic monitoring, industrial automation, smart-campus systems, infrastructure monitoring, and other intelligent vision applications. Improved processing latency can help such systems respond more quickly to incoming information.

Safety

Reliable hardware operation is important when inference results are used in applications involving safety or critical decision-making. The system should include appropriate validation, error detection, memory protection, and reliable I/O mechanisms. Incorrect data transfer or processor execution should be detected before it affects the final application output.

Ethics

Camera-based systems should be developed with responsible data-handling practices. Privacy should be considered during data collection, processing, storage, and transmission. Access to processed data should be restricted to authorized users, and unnecessary collection of personal information should be avoided.

Economics

Although the advanced architecture requires additional hardware and control resources, its improved performance may justify the additional cost for high-throughput applications. Higher processing efficiency can allow more inference operations to be completed using the available computing resources.

Accessibility

The proposed architecture can help provide responsive intelligent-processing capabilities across different deployment environments. The system should be designed so that its performance remains reliable under different workload conditions and resource limitations.

Professional Responsibility

Engineers should clearly distinguish between analytical calculations, simulation results, and actual hardware measurements. Assumptions, input parameters, limitations, and evaluation methods should be documented properly. This improves the reliability, transparency, and reproducibility of the proposed architecture.

8. Conclusion

Summarize:

- Proposed solution
- Major findings

- Achievement of requirements
- Limitations
- Possible improvements

The analysis demonstrates that processor architecture has a significant effect on the performance of cloud-based inference workloads. Pipeline organization, instruction-level parallelism, cache behaviour, memory access, and DMA transfer efficiency all influence the final execution time.

The comparison showed that Alternative B provides better overall performance because it can execute independent instructions in parallel and overlap computation with memory and I/O operations. Its lower CPI and stall count indicate more efficient utilization of processor resources.

The calculated results confirm that both alternatives satisfy the required inference latency of less than 2.0 ms per batch. Both architectures also provide sufficient DMA throughput to support the required aggregate camera input rate of 48 MB/s.

Alternative B achieved a CPI of 1.205 and an inference latency of approximately 0.502 ms, while Alternative A achieved a CPI of 1.380 and a latency of approximately 0.575 ms. Therefore, Alternative B provides a measurable improvement within the analytical model.

The design also demonstrates the importance of balancing performance with implementation complexity. Although Alternative B requires more advanced hardware mechanisms, its benefits become more valuable for workloads with multiple simultaneous streams and high computational requirements.

Future improvements can include detailed processor simulation, hardware prototyping, power analysis, energy-per-inference measurement, larger cache studies, and testing with increased numbers of camera streams.

9. Student Reflection

- What did you learn from this assignment beyond what was directly taught in the classroom?
- If you were given additional time/resources, what would you improve and why?

Through this assignment, I gained a better understanding of how computer architecture concepts are applied to a practical engineering problem. I learned that processor performance depends on several interacting factors rather than on clock frequency alone.

I understood how instruction count, CPI, pipeline stalls, cache misses, memory delays, and DMA throughput can be used together to evaluate an architecture. This helped me connect theoretical concepts with practical performance analysis.

The comparison of Alternative A and Alternative B also helped me understand the trade-off between simple hardware and high-performance hardware. A more complex architecture can provide better parallelism, but it also requires additional control logic and hardware resources.

Another important learning outcome was understanding the role of memory and I/O systems in processor performance. Even when the processor is fast, cache misses, memory latency, or insufficient I/O bandwidth can become system-level bottlenecks.

I also improved my ability to present quantitative results using tables, calculations, and performance metrics. This assignment strengthened my analytical and engineering decision-making skills.

If more resources were available, I would perform a cycle-accurate simulation of both architectures and compare their performance under different workloads. I would also investigate energy consumption, thermal behaviour, hardware area, and performance as the number of camera streams increases.

10. References

Use an appropriate citation format and acknowledge all external sources, datasets, standards, software and AI-assisted tools used.

1. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann.
2. D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.
3. M. Mano and M. D. Ciletti, *Computer System Architecture*, Pearson.
4. MathWorks, *MATLAB Documentation*, MathWorks.
Used for numerical computation, performance analysis, and graph generation.
5. MathWorks, *Simulink Documentation*, MathWorks.
Used as a reference for simulation and system modelling concepts where applicable.
6. Git Documentation, *Git Reference Documentation*.
Used for source-code version control and project management.
7. GitHub Documentation, *GitHub Documentation*.
Used for repository management and submission of source/configuration files.
8. **AI-Assisted Tool:** OpenAI, ChatGPT, used as an AI-assisted tool for organizing report content, explaining computer architecture concepts, and assisting with MATLAB/Python code development. AI-assisted content was reviewed and adapted for the requirements of this assignment.

Hennessy, J. L., & Patterson, D. A., *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann.

9. Patterson, D. A., & Hennessy, J. L., *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.

10. Mano, M. M., & Ciletti, M. D., *Computer System Architecture*, Pearson.

11. MathWorks, *MATLAB Documentation*, MathWorks.

12. MathWorks, *Simulink Documentation*, MathWorks.

13. Git, *Git Reference Documentation*, *Git Documentation*.

GitHub, *GitHub Documentation*, *GitHub*.

15. OpenAI, *ChatGPT*, AI-assisted tool used for understanding computer architecture concepts, organizing report content, and assisting with Python/MATLAB code development.

Github:

<https://github.com/Nivedha0306/M.NIVEDHA-192471040--CA>

B. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

C. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

D. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action